Iris on Indirection Theory: Lightweight Higher-order Ghost State

Anonymous Author(s)

Abstract

Today, semantic models of languages with concurrency and higher-order state are constructed using step-indexed predicates over resources and higher-order ghost state. There are two main lines of work for constructing such models. On the one hand, Iris provides these in full generality, at the cost of a large body of mechanised domain theory that is difficult to learn and port to new proof assistants. On the other hand, Indirection Theory is much simpler through the use of ordinary dependent types, but its foundation in separation algebra limits the expressiveness of its resource model compared to Iris-style higher-order ghost state. We resolve this conundrum by showing that Indirection Theory can be as expressive as Iris. To that end, we give a lightweight mechanisation of Indirection Theory in Lean, generalised so knots contain resources from an Iris-style resource algebra functor. We then reconstruct the Iris base logic on top of it. The advantage of this approach is in the elementary metatheory: a user can follow the whole construction without domain theory. We conduct two case studies to show that the base logic is expressive and faithful to Iris: a biorthogonal logical relation for higher-order state, and an Iris-style program logic with invariants and user-definable ghost state, complete enough to verify concurrent and higher-order programs.

Keywords: separation logic, step-indexing, indirection theory, higher-order ghost state, Iris, Lean

1 Introduction

In the semantics of programming languages, one often encounters definitions that are apparently circular, and hence not well-founded. Step-indexing breaks this circularity by stratifying the definitions by a natural number *index*.

For example, to define a (binary) logical relation for the untyped lambda calculus, we can state the relation on values $\mathcal{V}$ as follows ($\mathcal{E}$ is the relation on expressions).

$$\mathcal{V}_n \triangleq \{(\lambda x.\, e_1, \lambda x.\, e_2) \mid \forall\, k < n.\\ \forall\, (v_1, v_2) \in \mathcal{V}_k.\, (e_1[v_1/x], e_2[v_2/x]) \in \mathcal{E}_k\}$$

Having the recursive occurrences at $k < n$ makes the definition well-founded.

Despite its elegance and conceptual simplicity, explicit low-level index manipulation does not scale well, obscuring proofs with index arithmetic. Modern presentations thus hide indices behind modal logics [13, 6], which provide essential high-level facilities such as defining guarded recursive predicates, and reasoning by Löb induction. Lightweight implementations of these modal logics exist. In IxFree [26], for example, one can write the following definition instead, using the $\triangleright$ (later) modality.

$$\mathcal{V} \triangleq \{(\lambda x.\, e_1, \lambda x.\, e_2) \mid\\ \forall\, (v_1, v_2) \in \triangleright\, \mathcal{V}.\, (e_1[v_1/x], e_2[v_2/x]) \in \triangleright\, \mathcal{E}\}$$

For this calculus, the domain of step-indexed propositions that IxFree provides is sufficient. For more advanced language features such as state and concurrency, however, we would need a domain of *propositions over heaps*, and even defining one is a problem. A classic example of this occurs in a language with first-class locks [15], each guarding a *resource invariant* [25], a proposition describing the resources the lock protects. A naive model may be given as follows.

$$\mathit{Heap} \triangleq \mathit{Loc} \rightharpoonup_{\text{fin}} \mathit{Res}$$
$$\mathit{Res} \triangleq \mathit{Val} \mid \mathit{Lock}\, \mathit{HProp}$$
$$\mathit{HProp} \triangleq \mathit{Heap} \to \mathit{Prop}$$

There is a circularity in the semantic domains: unfolding *Heap* and *HProp* in *Res* gives

$$\mathit{Res} \triangleq \mathit{Val} \mid \mathit{Lock}\, ((\mathit{Loc} \rightharpoonup_{\text{fin}} \mathit{Res}) \to \mathit{Prop})$$

where *Res* occurs negatively. This equation has no solution in the category of sets, and a proof assistant would reject it.

The problem of circular domain equations has been extensively studied, and has led to two main lines of work. The more well-known one is Iris [17], which solves the equations in the category of *complete ordered families of equivalences* (COFEs). The other is Indirection Theory, which uses "only simple, elementary mathematics" [16], i.e., ordinary dependent types, to construct a *knot* containing approximations of predicates.

Researchers building on these frameworks today face a tradeoff. Due to its maturity and generality, Iris is a large and complex framework, and working at its core demands deep

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

expertise. The developers of PulseCore noted the requirement for "considerable expertise and effort in mechanized domain theory" [14] when considering a port to F★. A faithful port such as Iris-Lean [18, 12] is a similarly substantial development of about 60K lines of code. In contrast, Indirection Theory is substantially simpler (the original construction is only 400 LoC [16], though that is the core alone, without a base logic, ghost state or a program logic), and has been used successfully in the Verified Software Toolchain (VST) [3, 4] and more recently in PulseCore [14]. However, it is built on *separation algebra*, which restricts the model from having the full expressiveness of Iris's higher-order ghost state: Indirection Theory-based VST "could express only limited higher-order ghost state", motivating a reimplementation on top of Iris [24], and PulseCore implements "two primitive kinds of higher order ghost state: invariants and predicates", relying on encodings for the others [14]. Whether these limitations are inherent or an artifact of these developments is unclear.

We show that the limitations are not inherent. To that end, we develop a *lightweight* mechanisation of Indirection Theory in Lean [11], generalised to carry resources from a *resource algebra functor*. It is lightweight in that the metatheory is elementary: a user can follow the whole construction without domain theory. We implement the Iris *base logic* on top of it, which is small yet expressive. The development is self-contained and compact, taking under 1,700 LoC.

To demonstrate that the base logic is faithful to Iris, we use it in two case studies: a biorthogonal logical relation for a heap-manipulating language, and a program logic for a concurrent, heap-manipulating one. The program logic is complete enough to verify concurrent and higher-order programs, as we show on several small examples.

In summary, we make the following contributions:

- A lightweight implementation of Indirection Theory in Lean, generalised to support user-definable higher-order ghost state, thus making the construction of higher-order semantic models accessible to a wider audience and clarifying the connection to Iris's machinery (§3)
- A case study in direct use of the library: a biorthogonal logical relation for a language with higher-order state (§4)
- A reconstruction of Iris-style higher-order ghost state, impredicative invariants, and weakest preconditions atop Indirection Theory, complete enough to verify concurrent and higher-order programs (§5)

2 Background

In this section, we recall the parts of Iris that our development reconstructs. Iris is a higher-order concurrent separation logic. Reasoning about programs with it is carried out using a program logic with advanced features, such as

$$
\begin{array}{lll}
P, Q ::= & \ulcorner \phi \urcorner \mid P \wedge Q \mid P \vee Q \mid P \Rightarrow Q & \\
& \mid \forall x.\, P \mid \exists x.\, P & \\
& \mid P * Q \mid P \mathrel{-\!\!*} Q & \text{(separation)} \\
& \mid \triangleright P \mid t \equiv u & \text{(step-indexing)} \\
& \mid \Box P \mid \mathit{Own}(a) \mid \Rrightarrow P & \text{(resources)}
\end{array}
$$

Figure 1. The primitive connectives of the Iris base logic.

invariants and *fancy updates*. These features are not primitive but derived [20] from a much smaller base logic, whose propositions have type *iProp* (§2.1). The base logic is in turn an instance of a more general base logic of resources, whose propositions have type *uPred* (§2.2). Constructing *iProp* and *uPred* requires Iris's advanced structures (§2.3); our aim is to construct the corresponding version of *iProp* through simpler structures (§2.4).

2.1 The base logic *iProp*

The Iris base logic [17] is a higher-order affine separation logic with two further groups of connectives (Figure 1). The first has to do with step-indexing: the later modality $\triangleright$ and an internalised equality $t \equiv u$. The second has to do with separation logic resources. $\mathit{Own}(a)$ asserts ownership of the resource a. The persistence modality $\Box P$ says that P holds of the *core* of the resource at hand, a duplicable part of it. The *basic update* $\Rrightarrow P$ says the resource at hand can be updated to one satisfying P, without invalidating any *framing* resource. We make this more precise in §2.3. Invariants, higher-order ghost state and fancy updates are derived from these.

2.2 From *uPred* to *iProp*

To see how *iProp* is constructed, we begin with *uniform predicates uPred*, predicates on a step index and a type of resource M.

$$
\begin{aligned}
\mathit{uPred}(M) \triangleq \{P : \mathbb{N} \to M \to \mathit{Prop} \mid \forall\, & n_1, n_2, x_1, x_2. \\
& n_2 \le n_1 \Rightarrow x_1 \preccurlyeq_{n_2} x_2 \Rightarrow P\, n_1\, x_1 \Rightarrow P\, n_2\, x_2\}
\end{aligned}
$$

Intuitively, uniform predicates remain true (1) when there are fewer steps left to observe, and (2) on larger resources. The first condition makes them *downward-closed* in the step index: a *uPred* remains true one step later, i.e. $P \vdash \triangleright P$. The second condition makes them *upward-closed* under resource extension $\preccurlyeq_n$, in turn making them affine, i.e. $P * Q \vdash P$.

To reach *iProp*, we need to instantiate *uPred* with some type of resource M, which we have thus far left abstract. As a first approximation, we give three requirements on M.

Firstly, it must have enough structure to interpret the connectives of Figure 1: composition $\cdot$ to split resources for $*$, a duplicable *core* for $\Box$, and a notion of when a resource is valid to ensure $\Rrightarrow$ does not invalidate frames. Iris packages these properties as the notion of a *unital camera*; more detail on this is given in §2.3. Secondly, M must be able to contain

propositions *iProp*, which we have not yet defined. Therefore, what we instantiate M with must be defined mutually recursively with *iProp*.

Given M is a user-chosen *unital camera functor* F, Iris takes the fixed point of the following equations.

$$iRes \cong F(iProp) \qquad iProp \triangleq uPred(iRes)$$

Unfolding, this is $iRes \cong F(uPred(iRes))$, a recursive domain equation. Its solution gives us *iProp*.

2.3 Constructing *uPred*: COFEs and UCMRAs

Making §2.2 precise requires a substantial amount of structure, almost all of it for solving the domain equation. We recall it briefly, to fix terminology, and because §3.6 compares it against what Indirection Theory needs.

Because of cardinality, no type satisfies the equation as given in the category of sets. The idea of the solver is to move to a setting of types with approximate equality, under which two elements are considered equal up to n steps if they cannot be distinguished within that many steps. A solution is then a type isomorphic to its own unfolding with respect to approximate equality. An OFE provides a canonical structure for capturing this notion of approximation. Cameras then equip resources with an OFE structure. The OFE structure serves purposes beyond solving the recursive domain equations: in particular, the stronger COFE structure supports Löb induction and guarded recursive definitions.

(C)OFEs. An *ordered family of equivalences* is a type A with an equivalence $\equiv$ and a family of equivalences $(\equiv_n)_{n\in\mathbb{N}}$, satisfying $x \equiv y \Leftrightarrow \forall n.\, x \equiv_n y$ and $n \leq m \Rightarrow x \equiv_m y \Rightarrow x \equiv_n y$. Every type carrying step-indexed data must be an OFE, and every function between such types must be *non-expansive*: $x \equiv_n y \Rightarrow f(x) \equiv_n f(y)$. It is *contractive* if the weaker $\forall m < n.\, x \equiv_m y$ already gives $f(x) \equiv_n f(y)$. An OFE may further be *complete* (a COFE), which gives contractive functions unique fixed points, allowing guarded recursive definitions.

Resource algebras and (unital) cameras. A type of resources satisfying the requirements of §2.2 is called a *resource algebra* (RA). Concretely, it is a type with a composition operation $\cdot$ which combines two resources, a predicate $\checkmark$ which says whether a resource is valid, and a partial *core* operation *pcore* which returns the duplicable part of a resource when it has one. A *camera* (CMRA) is a resource algebra whose carrier is an OFE, so that its resources inherit the corresponding notion of approximate equality. In addition, a camera is equipped with step-indexed validity $\checkmark_n$, which can be viewed as the validity analogue of step-indexed equality. A *unital* camera (UCMRA) has an additional empty resource ε.

Functors and the solver. Since resources store propositions, the user does not supply a unital camera but a *unital camera functor* F: given a COFE of propositions, F builds the unital camera of resources storing them. F must be *locally contractive*, meaning that resources built from propositions equal up to n steps are equal up to $n+1$ steps. The *solver* is the theorem that such an F admits the fixed point of §2.2, a COFE X with $X \cong uPred(F(X))$.

Löb induction and guarded recursive definition. In Iris, *iProp* is a COFE, which allows us to reason by Löb induction; under the hood, this corresponds to strong induction on the step-index. *iProp* also supports guarded recursive definitions. In such definitions, recursive occurrences typically appear under the $\triangleright$ modality, which ensures that they are contractive and hence admit a unique fixed point.

2.4 An alternative route to *iProp*

The Iris base logic (§2.1 and Figure 1) is expressive, well-understood, and widely used. We thus take *iProp* as a specification, viewing the construction of §2.2 and §2.3 as one way to satisfy it. We then explore the use of Indirection Theory [16] as an alternative way to realise this base logic, in a form that suffices for common applications such as logical relations (§4) and program logics (§5). The next section (§3) gives the construction, concluding with a comparison to Iris (§3.6).

3 The Indirection Theory Library

In this section, we construct a replacement for the base logic of Figure 1: the type of propositions $HPred(M)$, standing for *hereditary predicates*, over a user-chosen type of resources M. This is done in four layers: resources (§3.1), knots (§3.2), hereditary predicates (§3.3), and logical connectives (§3.4). §3.5 then discusses two important features of our step-indexing model: Löb induction and guarded recursive definitions. Finally, §3.6 concludes the section by comparing our construction with Iris's.

3.1 Resources

The logic we are building is a separation logic. We thus begin by recalling the notion of a resource algebra, an abstraction of the resources the propositions of our logic describe. An RA is a type α of resources equipped with a composition operator $\cdot : \alpha \to \alpha \to \alpha$, a validity predicate $\checkmark : \alpha \to Prop$, and an operation $pcore : \alpha \to Option\,\alpha$ ("partial core"), satisfying the laws of Figure 2. Resource algebra *inclusion* is defined as $a \preccurlyeq b \triangleq \exists c.\, b = a \cdot c$.

$$\begin{array}{l}
(a \cdot b) \cdot c = a \cdot (b \cdot c) \\
a \cdot b = b \cdot a \\
\checkmark (a \cdot b) \Rightarrow \checkmark a \\
pcore\, a = some\, c \Rightarrow c \cdot a = a \\
pcore\, a = some\, c \Rightarrow pcore\, c = some\, c \\
a \preccurlyeq b \Rightarrow pcore\, a = some\, c_a \Rightarrow \\
\quad \exists c_b.\ pcore\, b = some\, c_b \wedge c_a \preccurlyeq c_b
\end{array}$$

Figure 2. Resource algebra laws.

Composition is associative and commutative. Validity is closed "downward", so subparts of a valid resource are also valid. Some resources may be duplicated and shared freely: a is *duplicable* if $a \cdot a = a$. $pcore\, a$ is the duplicable part of a, if it exists. The fourth law says a contains its core, and the fifth that the core is its own core. Both of these together imply that the core is duplicable. The last law says that the core is monotone: if a is included in b, the core of a is included in the core of b.

A separation logic also needs a resource standing for ownership of nothing. A *unital resource algebra* (URA) is a resource algebra with such an element, $\varepsilon : \alpha$. The unit is valid ($\checkmark\ \varepsilon$), is the identity under composition ($\varepsilon \cdot a = a$), and is its own core. The unit is included in every resource, so by monotonicity every element of a URA has a core, which we denote by $core\, a$.

In a separation logic with higher-order ghost state, a resource may contain a proposition of the logic, which itself describes a resource. To break the cycle, we must abstract over the type P of propositions. Later, we will tie the knot by choosing P to be the type of propositions being defined.

Next, it is not enough to only parameterise resources over P, as we will need to work with *approximations* of the propositions stored in them. We must be able to weaken these approximations without changing the structure of a resource. This leads us to the (new) notion of a *resource algebra functor* (RAF), a type $M : Type \to Type$. $M(P)$ is the type of resources containing propositions P, and has an operation $map : (P \to Q) \to M(P) \to M(Q)$ satisfying the laws of Figure 3, which are that map is functorial (first two) and commutes with all the resource algebra operations (next three).

$$
\begin{aligned}
&map\ id\ a = a \\
&map\ (g \circ f)\ a = map\ g\ (map\ f\ a) \\
&map\ f\ (a \cdot b) = map\ f\ a \cdot map\ f\ b \\
&\checkmark\ a \Rightarrow \checkmark\ (map\ f\ a) \\
&pcore(map\ f\ a) = Option.map\ (map\ f)\ (pcore\ a) \\
&map\ f\ a = x \cdot y \Rightarrow \checkmark\ (x \cdot y) \Rightarrow \\
&\quad \exists\, x'\ y'.\ a = x' \cdot y' \wedge map\ f\ x' = x \wedge map\ f\ y' = y
\end{aligned}
$$

Figure 3. Resource algebra functor laws.

A *unital* resource algebra functor is simply a resource algebra functor M where $M(P)$ is a unital resource algebra. The last law of Figure 3 lets us derive that map preserves the unit, i.e. $map\ f\ \varepsilon = \varepsilon$, which would otherwise have to be assumed.

3.2 Knots

We now have resources that store propositions of an arbitrary type P, but not yet the type of propositions itself.

Propositions should be predicates on resources, and resources should store propositions, so the type P we want would have to satisfy $P \cong M(P) \to Prop$. This has no solution in set theory. Iris solves a variant of this equation in the category of COFEs. We will take a more elementary route, based on the idea of working with *approximations* of propositions.

The idea is to assign every proposition a *level* $n \in \mathbb{N}$. A proposition of level n+1 is a predicate on resources storing propositions of level n. At level 0, there are no lower levels to refer to, so there is a single trivial proposition. Intuitively, the level says how good an approximation a proposition is, or how much information it has. More concretely, it measures how deeply a proposition can inspect the propositions stored in the resources it describes.

One more refinement is required. A proposition true at level n+1 must remain true at every lower level, as it contains "more" information. However, a predicate on level-n resources says nothing about resources at lower levels. We therefore pair each predicate with its own "truncation" to the level below, so that a level-n proposition is really a sequence of n predicates of successively smaller levels.

We now make this precise, building up to the propositions of the logic (§3.3). Let M be a (user-chosen) resource algebra functor. We write $Res(P) \triangleq \{a : M(P) \mid \checkmark\ a\}$ for a valid resource of M. We then define a type of "raw" predicates *at* a level:

$$
\begin{aligned}
RawPredAt_0 &\triangleq unit \\
RawPredAt_{n+1} &\triangleq RawPredAt_n \times (Res(RawPredAt_n) \to Prop)
\end{aligned}
$$

The first element of the pair in the recursive case is the "truncation" of the predicate, while the second element is the predicate itself. Thus these predicates are over resources containing the (raw) predicates of the level below, the first step towards tying the knot as promised earlier.

RawPredAt is a type family, i.e. with one type per level, but we would like a type of propositions usable at every level. The next step towards that is to hide the level using a dependent pair. A *knot* is hence a level and a valid resource of that level.

$$Knot \triangleq \Sigma\, (n : \mathbb{N}).\ Res(RawPredAt_n)$$

For a knot k, we write $|k|$ for its level n and $res(k)$ for its resource. We also name the type of predicates over knots.

$$RawPred \triangleq Knot \to Prop$$

The two types are related by a projection and an embedding. $stratify_n$ projects a predicate over knots onto a level-n one, retaining its value at every knot k with $|k| < n$. $unstratify_n$ embeds a level-n predicate as a predicate over knots that is false at levels $\geq n$. The two operations satisfy

$$
\begin{aligned}
&stratify_n \circ unstratify_n = id \\
&|k| < n \Rightarrow unstratify_n(stratify_n\ P)\ k = P\ k
\end{aligned}
$$

Differences from the original construction [16] are in the pervasive use of RA validity, and the use of a concrete knot and stratify/unstratify, rather than the abstract squash/unsquash.

3.3 Hereditary predicates

A proposition true of a knot should remain true as the knot is approximated further or gains resources. We laid some groundwork for the former earlier in *RawPredAt*, but nothing yet ensures that a predicate over knots (*RawPred*) will respect either. We now define these two ways in which knots may change. The predicates closed under them, *hereditary predicates*, will be the propositions of the logic.

Aging. The first way a knot changes is by *aging*, which approximates it one level further. A proposition that holds of a knot should continue to hold once the knot is aged, since an aged knot carries strictly less information. Because the level strictly decreases, recursion through stored propositions is well-founded, which is what gives the logic $\triangleright$ and Löb induction.

Aging a knot from level $n{+}1$ to n replaces every stored proposition by its truncation. Truncation of a $\mathit{RawPredAt}_{n+1}$ simply projects its first component (π_1). Because a knot contains a resource algebra functor, we can use its *map* to age all its stored propositions. We can therefore define the function $\mathit{age}_1 : \mathit{Knot} \to \mathit{Option}\,\mathit{Knot}$ as follows.

$$\mathit{age}_1\,(n{+}1, \phi) \triangleq \mathit{some}(n, \mathit{map}\,\pi_1\,\phi) \qquad \mathit{age}_1\,(0, \phi) \triangleq \mathit{none}$$

We write $k \downarrow k'$ to mean "k aged by one level is k'", i.e. $\mathit{age}_1\,k = \mathit{some}\,k'$. $\downarrow^*$ and $\downarrow^+$ are its reflexive-transitive and transitive closures. $\downarrow^+$ strictly decreases the level, while $\downarrow^*$ never increases it.

Resource extension. The second way a knot changes is by gaining resources. A proposition that holds of a resource should continue to hold of a larger resource, which is what makes the logic affine and validates the frame rule.

Concretely, a knot may gain a frame at its own level.

$$k \preccurlyeq k' \triangleq |k| = |k'| \land \mathit{res}(k) \preccurlyeq \mathit{res}(k')$$

Composing the two gives us a monotonicity relation:

$$k \sqsubseteq k' \triangleq \exists k''.\, k \downarrow^* k'' \land k'' \preccurlyeq k'$$

Hereditary. A predicate is *hereditary* when it is closed under $\sqsubseteq$.

$$\mathit{hereditary}\,p \triangleq \forall k\,k'.\, p\,k \Rightarrow k \sqsubseteq k' \Rightarrow p\,k'$$
$$\mathit{HPred} \triangleq \{p : \mathit{RawPred} \mid \mathit{hereditary}\,p\}$$

$\preccurlyeq$ was not part of the original construction [16], but is present as the join order $\leq$ in MSL [4]. $\sqsubseteq$ is Iris's *uPred* monotonicity condition.

3.4 The base logic

Having defined knots and hereditary predicates, we can finally define the connectives of the base logic (Figure 4).

$$\begin{array}{ll}
P \vdash Q & \triangleq \forall k.\, P\,k \Rightarrow Q\,k \\
(P \land Q)\,k & \triangleq P\,k \land Q\,k \qquad (\forall x.\, P\,x)\,k \triangleq \forall x.\, P\,x\,k \\
(P \lor Q)\,k & \triangleq P\,k \lor Q\,k \qquad (\exists x.\, P\,x)\,k \triangleq \exists x.\, P\,x\,k \\
(P \Rightarrow Q)\,k & \triangleq \forall k'.\, k \sqsubseteq k' \Rightarrow P\,k' \Rightarrow Q\,k' \\
(P * Q)\,k & \triangleq \exists x_1\,x_2.\, \mathit{res}(k) = x_1 \cdot x_2 \land P\,(|k|, x_1) \land Q\,(|k|, x_2) \\
(P \mathrel{-\!\!*} Q)\,k & \triangleq \forall k'\,z.\, k \downarrow^* k' \Rightarrow \checkmark(\mathit{res}(k') \cdot z) \Rightarrow \\
 & \qquad P\,(|k'|, z) \Rightarrow Q\,(|k'|, \mathit{res}(k') \cdot z) \\
(\Rrightarrow P)\,k & \triangleq \forall k'\,z.\, k \downarrow^* k' \Rightarrow \checkmark(\mathit{res}(k') \cdot z) \Rightarrow \\
 & \qquad \exists x.\, \checkmark(x \cdot z) \land P\,(|k'|, x) \\
(\Box P)\,k & \triangleq P\,(|k|, \mathit{core}(\mathit{res}(k))) \\
(\triangleright P)\,k & \triangleq \forall k'.\, k \downarrow^+ k' \Rightarrow P\,k' \\
(P \equiv Q)\,k & \triangleq \forall k'.\, |k'| \leq |k| \Rightarrow (P\,k' \Leftrightarrow Q\,k') \\
\mathit{Own}(a)\,k & \triangleq a_{|k|} \preccurlyeq \mathit{res}(k)
\end{array}$$

Figure 4. Definition of the connectives of the base logic.

Entailment, conjunction, disjunction, and quantifiers are pointwise. Implication quantifies over the $\sqsubseteq$-successors of k, which is what makes it hereditary. Separating conjunction splits the resource at the knot's own level. The magic wand quantifies over $\downarrow^*$-successors and over frames, again to be hereditary. The basic update modality is the primitive for updating ghost resources in the presence of a frame. Persistence evaluates its argument at the core of the resource. Later evaluates its argument at every knot strictly below the current level. Internal equality of propositions holds at k when both agree at every knot at or below k's level. The except-0 modality $\diamond P \triangleq \triangleright \mathit{False} \lor P$, meaning "$P$ at every level above 0", is derived, and is used in §5.

Ownership is the only construct that differs significantly from Iris: Iris's $\mathit{Own}(a)$ takes a fixed resource a, but here the resource depends on the level, so a single a does not suffice. Here, *Own* takes instead a *coherent family* of resources $a : \forall n.\, M(\mathit{RawPredAt}_n)$: one resource per level, with a coherence condition saying that it is compatible with aging:

$$\mathit{map}\,\pi_1\,(a_{n+1}) = a_n$$

This is a sufficient condition to prove that *Own* is hereditary. The definition in Figure 4 reads: $\mathit{Own}(a)$ holds at a knot k when the resource at k's level, $a_{|k|}$, is included in $\mathit{res}(k)$.

We also overload the notation $\mathit{Own}(a)$ for the case where a is an element of a resource algebra A, rather than a coherent family. A resource algebra A can be turned into a resource algebra functor using the *constant* resource algebra functor

$$C(A)(P) \triangleq A$$

whose *map* ignores its first argument. Coherence is then trivial by the definition of C. We thus write $\mathit{Own}(a)$ for $\mathit{Own}(\lambda n.\, a)$ when a comes from a resource algebra embedded via a constant functor; §4 and §5 both rely on this for the resources that do not depend on the level.

3.5 Löb induction and guarded recursion

The logic of hereditary predicates enjoys Löb induction, strong induction on the level of the knot, as well as guarded recursion. Definitions such as the weakest precondition of §5 are recursive, and are taken as the fixed point of a contractive function $f : (A \to \mathit{HPred}) \to (A \to \mathit{HPred})$ on families of propositions indexed by a type A (the arguments of the recursive definition).

$$\begin{aligned}&\mathit{contractive}\, f \triangleq \forall P, Q, k.\\ &\quad (\forall x, k'.\, |k'| < |k| \Rightarrow (P\, x\, k' \Leftrightarrow Q\, x\, k')) \Rightarrow\\ &\quad\quad \forall x.\, f\, P\, x\, k \Leftrightarrow f\, Q\, x\, k\end{aligned}$$

This intuitively means that recursive occurrences of the family being defined are guarded by $\triangleright$. Every contractive f has a fixed point, $\mathit{Rec}\, f = f(\mathit{Rec}\, f)$, obtained by recursion on the level of the knot.

3.6 Comparison with Iris

The differences between our construction and Iris's have been discussed throughout §2 and this section. We recap them here.

Solving the domain equation. Iris defines $\mathit{uPred}(M)$ for any unital camera M, and obtains iProp by solving $\mathit{iRes} \cong F(\mathit{uPred}(\mathit{iRes}))$ (§2.3). Requiring a single type isomorphic to its own unfolding is what necessitates the category of COFEs and the solver of America and Rutten [2]. We define HPred for any unital resource algebra functor M, and obtain the resources of each level n by recursion on $\mathbb{N}$, storing the propositions of level $n-1$. Requiring a type per level is what allows us to get away with an ordinary recursive definition, without the solver, at the cost of the level being part of every knot.

Propositions. Iris propositions are predicates on $\mathbb{N} \times M$, so the index and the resource are independent. Lowering the index leaves the resource unchanged. Ours are predicates on Knot, so the resource depends on the level. Lowering the level maps the resource by $\mathit{map}\, \pi_1$.

Resources. The user of Iris supplies a unital camera functor, so resources are OFEs and have step-indexed equality $\equiv_n$ and validity $\checkmark_n$. The user of our logic supplies a unital resource algebra functor (§3.1) instead, so resources only have ordinary equality and a single validity predicate.

Ownership. Iris's $\mathit{Own}(a)$ takes a single $a : \mathit{iRes}$, since the type of a resource does not depend on the index. The same a is used at every index. Ours takes a coherent family (§3.4), since the type of a resource depends on the level. The resource a_n is used at level n.

Internal equality. Iris's internal equality works on both propositions and resources, as they are both OFEs. Ours works on propositions only, with first-order resources handled with Leibniz equality.

4 Case Study: Logical Relations

Our first case study uses the library directly, without the program logic of §5. We formalise a binary logical relation for an untyped, call-by-value, *stateful* lambda calculus, extending the biorthogonal construction of Balik et al. [7] with a heap.

$$\begin{aligned}e &::= x \mid v \mid e\, e \mid \mathit{ref}\, e \mid !e \mid e := e \mid \mathit{free}\, e\\ v &::= \lambda x.\, e \mid \ell\\ K &::= \boxdot \mid K\, e \mid v\, K \mid \mathit{ref}\, K \mid !K \mid K := e \mid v := K \mid \mathit{free}\, K\end{aligned}$$

A heap $h \in \mathit{Loc} \rightharpoonup_{\mathrm{fin}} \mathit{Option}\, v$ is a finite, partial mapping of *locations* to *live cells* or *tombstones* (which represent deallocated locations). For a location $\ell \in \mathrm{dom}(h)$, $h(\ell) = \mathit{none}$ is a tombstone and $h(\ell) = \mathit{some}\, v$ is a live cell holding v. Allocation deterministically chooses a fresh location, dereferencing and updating require a live cell, and deallocation replaces a live cell with a tombstone. Configurations $c ::= \langle h, e\rangle$ are pairs of a heap and a closed expression. Evaluation contexts K specify left-to-right call-by-value evaluation. We write $K[e]$ for the expression obtained by filling the hole of K with e.

4.1 Biorthogonal logical relations

A binary logical relation is a useful device for proving *contextual equivalence*, a strong form of program equivalence under which no enclosing program context can distinguish two expressions by their observable behaviour. The observable behaviour we use here is *termination*: both expressions either terminate or do not terminate. To capture this notion, we define an *approximation* relation over two configurations $\mathcal{L}(c_1, c_2)$ as follows. If the expression in c_1 is already a value, then the expression in c_2 must terminate. If c_1 instead takes a step to c_1', $\mathcal{L}(c_1', c_2)$ must hold later. The *observation* relation O is $\mathcal{L}$ in both directions.

$$\begin{aligned}\mathcal{L}(c_1, c_2) &\triangleq \mathit{value}(c_1) \Rightarrow \mathit{terminates}(c_2)\\ &\qquad \wedge \forall c_1'.\, c_1 \longrightarrow c_1' \Rightarrow \Rightarrow\!\!\!\triangleright\, \mathcal{L}(c_1', c_2)\\ O(c_1, c_2) &\triangleq \mathcal{L}(c_1, c_2) \wedge \mathcal{L}(c_2, c_1)\end{aligned}$$

The recursive occurrence of $\mathcal{L}$ is guarded by $\triangleright$, so its body is contractive, allowing $\mathcal{L}$ to be defined using the library's fixed-point operator (§3.5). The basic update permits ghost state to be updated between operational steps. We use ghost state to define the notion of *related heaps* next.

4.2 Related heaps

To relate two heap-manipulating programs, we must relate the two heaps they execute on. Intuitively, the locations in the two heaps should correspond, so we maintain a finite, partial bijection T from locations in the left heap to locations in the right.

We record this bijection using the *authoritative map* resource algebra, which supports reasoning about a shared map. One owner may hold the complete map as the authoritative source of truth, while others hold *fragments*: knowledge

of specific entries in the map. Fragments can be duplicated and distributed freely, but they must all remain compatible with the authoritative map; an update of the authoritative map must therefore preserve every fragment.

$$\mathit{AuthMap}(A) \triangleq \mathit{Option}\big(\mathit{Ex}(\mathit{Loc} \rightharpoonup_{\text{fin}} A)\big) \times (\mathit{Loc} \rightharpoonup_{\text{fin}} A)$$

$$\bullet\, f \triangleq (\mathit{some}(\mathit{ex}(f)), \emptyset) \qquad \circ\, g \triangleq (\mathit{none}, g)$$

It is represented as a pair of an *exclusive*, optional map and a part of that map. Exclusivity ensures that there can be at most one authoritative map at a time. An element of this resource algebra is valid if the fragment component is included in the authoritative component, whenever the authoritative component is present. $\bullet\, f$ and $\circ\, g$ denote full and fragmentary ownership of f and g respectively.

To allow several owners to agree on a single value, we use the *agreement* resource algebra $\mathit{Ag}(A)$. Any number of owners may hold $\mathit{ag}(a)$ for a value a and freely duplicate it, but two such instances are valid only if they refer to the same value.

$$\checkmark(\mathit{ag}(a) \cdot \mathit{ag}(b)) \Leftrightarrow a = b$$

The bijection is tracked using an authoritative map of agreements of locations $\mathit{AuthMap}(\mathit{Ag}(\mathit{Loc}))$. $\mathit{tieAuth}(T)$ asserts ownership of the authoritative finite map recording the mapping, and fragment assertions $\mathit{tied}(\ell_1, \ell_2)$ record that $T(\ell_1) = \ell_2$.

$$\mathit{tieAuth}(T) \triangleq \mathit{Own}(\bullet\, T^{\mathit{ag}})$$
$$\mathit{tied}(\ell_1, \ell_2) \triangleq \mathit{Own}(\circ\, [\ell_1 \mapsto \mathit{ag}(\ell_2)])$$

where T^{ag} is the pointwise lifting of T to agreement values. The use of $\mathit{Ag}(\mathit{Loc})$ ensures that two fragments for the same left location agree on the right location.

The mapping between locations must be well-formed. We state this using an assertion $\mathit{tieWF}(T, h_1, h_2)$, meaning that T is injective (required for the map to be a bijection), tied locations occur in their respective heaps, and both cells are either live or deallocated.

$$\begin{aligned}&\mathit{tieWF}(T, h_1, h_2) \triangleq \\ &\quad (\forall\, \ell_1, \ell_1', \ell_2.\, T(\ell_1) = \ell_2 \land T(\ell_1') = \ell_2 \Rightarrow \ell_1 = \ell_1') \\ &\quad \land\, (\forall\, \ell_1, \ell_2.\, T(\ell_1) = \ell_2 \Rightarrow \ell_1 \in \mathrm{dom}(h_1) \land \ell_2 \in \mathrm{dom}(h_2)) \\ &\quad \land\, (\forall\, \ell_1, \ell_2.\, T(\ell_1) = \ell_2 \Rightarrow (\mathit{live}(h_1, \ell_1) \Leftrightarrow \mathit{live}(h_2, \ell_2)))\end{aligned}$$

Given a candidate value relation $\mathcal{V}$, related heaps must also contain related *values* at every pair of tied locations. We denote this condition by $\mathit{heapWF}(T, h_1, h_2)$.

$$\begin{aligned}&\mathit{heapWF}(T, h_1, h_2) \triangleq \forall\, \ell_1, \ell_2, v_1, v_2. \\ &\quad T(\ell_1) = \ell_2 \land h_1(\ell_1) = v_1 \land h_2(\ell_2) = v_2 \Rightarrow \triangleright\, \mathcal{V}(v_1, v_2)\end{aligned}$$

The heap relation $\mathcal{H}(h_1, h_2)$ is then the packaging of these three conditions.

$$\begin{aligned}&\mathcal{H}(h_1, h_2) \triangleq \exists\, T. \\ &\quad \mathit{tieAuth}(T) * \mathit{tieWF}(T, h_1, h_2) * \mathit{heapWF}(T, h_1, h_2)\end{aligned}$$

There are two complementary reasons for the $\triangleright$ in heapWF. Technically, $\mathcal{H}$ and $\mathcal{V}$ are defined mutually recursively: $\mathcal{H}$ depends on $\mathcal{V}$ through heapWF, while $\mathcal{H}$ is eventually used to define $\mathcal{V}$. The $\triangleright$ makes this recursive definition well-founded. Intuitively, this also makes sense because, if two locations are related, we must first spend one step reading the locations before we can inspect their contents and determine that the resulting values are related. Thus, the values obtained from related locations are related later.

$\mathcal{H}$ is preserved by all heap manipulations programs may perform.

$$\frac{\mathcal{H}(h_1, h_2) * \mathit{tied}(\ell_1, \ell_2) * \triangleright\, \mathcal{V}(v_1, v_2)}{\mathcal{H}(h_1[\ell_1 \mapsto v_1], h_2[\ell_2 \mapsto v_2])}\ \textsc{Update}$$

$$\frac{\mathcal{H}(h_1, h_2) * \mathit{tied}(\ell_1, \ell_2)}{\mathcal{H}(h_1[\ell_1 \mapsto \mathit{none}], h_2[\ell_2 \mapsto \mathit{none}])}\ \textsc{Free}$$

$$\frac{\begin{array}{c}\ell_i \notin \mathrm{dom}(h_i)\ (i \in \{1, 2\}) \\ \mathcal{H}(h_1, h_2) * \triangleright\, \mathcal{V}(v_1, v_2)\end{array}}{\Rrightarrow \big(\mathcal{H}(h_1[\ell_1 \mapsto v_1], h_2[\ell_2 \mapsto v_2]) * \mathit{tied}(\ell_1, \ell_2)\big)}\ \textsc{Alloc}$$

Updating tied locations requires the values they are updated with to be related one step later. We write $h[\ell \mapsto v]$ for an update of ℓ to a live cell containing v. Similarly, tied locations can be freed, which replaces both cells with the tombstone *none*. Only allocation of tied locations changes the ghost state: a basic update extends T and produces a new *tied* fragment.

4.3 The logical relation

We can now define the relations on contexts and expressions, which are also mutually recursive with $\mathcal{V}$. They depend on the observation relation $\mathcal{O}$, and so need $\mathcal{H}$ to talk about related heaps.

$$\begin{aligned}\mathcal{K}(K_1, K_2) &\triangleq \forall\, v_1, v_2.\, \mathcal{V}(v_1, v_2) \mathrel{-\!\!*} \\ &\qquad \forall\, h_1, h_2.\, \mathcal{H}(h_1, h_2) \mathrel{-\!\!*} \\ &\qquad \mathcal{O}(\langle h_1, K_1[v_1]\rangle, \langle h_2, K_2[v_2]\rangle) \\ \mathcal{E}(e_1, e_2) &\triangleq \forall\, K_1, K_2.\, \mathcal{K}(K_1, K_2) \mathrel{-\!\!*} \\ &\qquad \forall\, h_1, h_2.\, \mathcal{H}(h_1, h_2) \mathrel{-\!\!*} \\ &\qquad \mathcal{O}(\langle h_1, K_1[e_1]\rangle, \langle h_2, K_2[e_2]\rangle) \\ \mathcal{R}(e_1, e_2) &\triangleq \forall\, K_1, K_2.\, \triangleright\, \mathcal{K}(K_1, K_2) \mathrel{-\!\!*} \\ &\qquad \forall\, h_1, h_2.\, \mathcal{H}(h_1, h_2) \mathrel{-\!\!*} \\ &\qquad \mathcal{O}(\langle h_1, K_1[e_1]\rangle, \langle h_2, K_2[e_2]\rangle)\end{aligned}$$

$\mathcal{R}$ is $\mathcal{E}$ with a $\triangleright$ on $\mathcal{K}$. They differ in that $\mathcal{E}$ is the main expression relation and must also relate values, in which case no step occurs and $K_1[e_1]$ and $K_2[e_2]$ are evaluated *now*, so no later can be afforded. $\mathcal{R}$ is for expressions guaranteed

to take a step, and is used to define $\mathcal{V}$.

$$\begin{aligned}\mathcal{V}(\lambda x.\, e_1, \lambda x.\, e_2) &\triangleq \Box \forall v_1, v_2.\ \triangleright \mathcal{V}(v_1, v_2) \twoheadrightarrow \\ &\qquad \mathcal{R}((\lambda x.\, e_1)\, v_1, (\lambda x.\, e_2)\, v_2) \\ \mathcal{V}(\ell_1, \ell_2) &\triangleq \mathit{tied}(\ell_1, \ell_2) \\ \mathcal{V}(v_1, v_2) &\triangleq \mathit{False} \qquad \text{otherwise}\end{aligned}$$

For lambda abstractions, $\mathcal{V}$ relates those whose applications are in $\mathcal{R}$ when given $\triangleright\mathcal{V}$-related arguments. This relation must be persistent, since a pair of related lambda abstractions may be used any number of times. Additionally, the arguments can only be used after one step of β-reduction, so they only need to be related later. Locations are related if they are *tied*. All other mismatched value shapes are unrelated.

4.4 Open terms

To deal with open terms, we define a relation $\mathcal{G}$ on substitutions: two substitutions are related when they map every variable to $\mathcal{V}$-related values. $\mathcal{E}^\circ$ relates open terms by closing them under related substitutions.

$$\mathcal{G}(\gamma_1, \gamma_2) \triangleq \forall x. \mathcal{V}(\gamma_1(x), \gamma_2(x))$$
$$\mathcal{E}^\circ(e_1, e_2) \triangleq \forall \gamma_1, \gamma_2. \mathcal{G}(\gamma_1, \gamma_2) \twoheadrightarrow \mathcal{E}(e_1[\gamma_1], e_2[\gamma_2])$$

With the relations defined, we can state and prove the fundamental theorem.

$$\mathit{LocFree}(e) \Rrightarrow \mathit{True} \vdash \mathcal{E}^\circ(e, e)$$

The $\mathit{LocFree}(e)$ restriction reflects the fact that the fundamental theorem is stated for source programs. Source programs cannot contain literal locations, which only arise at runtime. Moreover, a literal location cannot be interpreted without a pair of heaps, which is not available in the definition above.

5 Case Study: Iris on Indirection Theory

In our second case study, we reconstruct an Iris-style program logic [17] using the base logic. We cover three main aspects: invariants, user-definable ghost state, and weakest preconditions. The object language is *HeapLang*, an untyped, call-by-value, concurrent lambda calculus with a heap. Its small-step operational semantics reduces a *thread pool* (a list of expressions) over a heap.

5.1 The ghost heap

The first thing we will need is an abstract heap to model various assertions about ownership of ghost state. We thus call this heap the *ghost heap*, and define it to be a finite map from ghost names to *cells*. The type of cells is parameterized by a resource algebra functor M.

$$\mathit{GHeap}(M)(P) \triangleq \mathit{Loc} \rightharpoonup_{\text{fin}} M(P)$$

$\mathit{GHeap}(M)$ is itself a unital resource algebra functor. We can thus instantiate *Knot* and *HPred* with $\mathit{GHeap}(M)$; we do so for the remainder of this section.

For convenience when working with GHeap, we define ownership of a single ghost cell by specialising $\mathit{Own}(\cdot)$ to a singleton heap. Given a location ι and a coherent family $a : \forall n. M(\mathit{RawPredAt}_n)$, the family $\lambda n. [\iota \mapsto a_n]$ is also coherent, and we define

$$\mathit{own}\, \iota\, a \triangleq \mathit{Own}((\lambda n \mapsto [\iota \mapsto a_n])).$$

We follow the same convention as for $\mathit{Own}(\cdot)$: we also write $\mathit{own}\, \iota\, a$ when a is an element of a resource algebra A, assuming that A has been lifted to a resource algebra functor via the constant functor C.

It remains to provide a usable type of cells. Intuitively, it is a sum of a number of resource algebras. We introduce each resource algebra as it is needed below, before defining their sum. A more precise definition can be found in the formal development. Our goal in this section is to provide intuition for how the ghost heap is organised and made extensible, and how it supports the resulting program logic.

5.2 The concrete heap

Our model must provide meaning to the points-to assertion $\ell \mapsto v$, which denotes ownership of a piece of the *concrete heap*, the actual heap that programs execute on. The concrete heap occupies a single cell of the ghost heap, at a distinguished location γ_{heap}. We formulate the concrete heap as a resource algebra $\mathit{Heap}(V)$ over a set of values V. We define it as a composition of resource algebras.

$$\mathit{Heap}(V) \triangleq \mathit{AuthMap}(\mathit{Ex}(V))$$

The authoritative component models the whole concrete heap, while fragments model points-to claims to individual cells. Since map values are exclusive, two points-to facts cannot refer to the same location. Concretely, points-to assertions $\ell \mapsto v$ represent ownership of a fragment

$$\ell \mapsto v \triangleq \mathit{own}\, \gamma_{\mathit{heap}}\, (\circ [\ell \mapsto \mathit{ex}(v)])$$

and the *state interpretation* $\mathit{SI}(\sigma)$ is a new assertion denoting ownership of the entire concrete heap σ

$$\mathit{SI}(\sigma) \triangleq \mathit{own}\, \gamma_{\mathit{heap}}\, (\bullet\, \sigma^{\mathit{ex}})$$

where σ^{ex} is its pointwise lifting to exclusive values. SI is used to define the weakest precondition later, but does not normally appear in user-level proofs.

The use of an authoritative map over exclusive values ensures that fragments are consistent with the whole heap.

$$\mathit{SI}(\sigma) * \ell \mapsto v \vdash \sigma(\ell) = v$$

Owning both $\mathit{SI}(\sigma) * (\ell \mapsto v)$ allows updates

$$\mathit{SI}(\sigma) * \ell \mapsto v \vdash \Rrightarrow \big(\mathit{SI}(\sigma[\ell := w]) * \ell \mapsto w\big)$$

and if $\ell \notin \mathit{dom}(\sigma)$, there is no compatible fragment mentioning ℓ, allowing allocation.

$$\mathit{SI}(\sigma) \vdash \Rrightarrow \big(\mathit{SI}(\sigma[\ell := v]) * \ell \mapsto v\big)$$

These rules capture precisely the intuition about concrete heap manipulation in a program, and they are used to prove weakest precondition rules for heap-manipulating operations.

5.3 Invariants

Next, our model gives meaning to *invariants*: named assertions whose resources are maintained throughout execution. An invariant body P can be any proposition. In particular, P may own resources from the concrete heap or ghost state. Invariants are moreover *impredicative*: P may itself mention invariants, including the invariant of which P is the body.

$\boxed{P}^{\iota}$ asserts that the invariant named by ι has body P. Invariants may be *allocated* from $\triangleright P$, yielding $\boxed{P}^{\iota}$ with a fresh name ι. They may be *opened* to obtain $\triangleright P$, and must later be *closed*, relinquishing the resources and reestablishing the invariant. An invariant cannot be opened while it is already open, for soundness.

To implement this, ghost heap cells must contain at least three more resource algebras. First, invariants themselves, where ι is a location in the ghost heap storing an approximation of P. Since P is a proposition, we use the resource algebra functor *InvF* to map its approximation to a resource algebra. Second, an invariant *registry*, the set of allocated invariant names, stored at the distinguished location γ_{reg}. Third, a *mask*, a finite or cofinite[1] set of invariant names.

$$\begin{aligned} \mathit{InvF}(P) &\triangleq \mathit{Ag}(P) \\ \mathit{Reg} &\triangleq \mathit{AuthMap}(\mathit{Ag}(\mathit{Unit})) \\ \mathit{Mask} &\triangleq \mathit{DisjMask}^{2} \end{aligned}$$

We can now understand the definition of $\boxed{P}^{\iota}$. It records the agreement that P is the invariant body stored at ι, as well as ownership of a fragment of the registry indicating that the name ι has been allocated.

$$\boxed{P}^{\iota} \triangleq \mathit{own}\,\iota\,\mathit{ag}(P) * \mathit{own}\,\gamma_{reg}\,(\circ\,[\iota \mapsto \mathit{ag}(\mathit{unit})])$$

We then define a new assertion *wsat* to keep track of which invariants are allocated and govern their allocation, opening, and closing. It plays an analogous "global" role to *SI*, and is likewise used to define the weakest precondition.

$$\begin{aligned} \mathit{wsat} &\triangleq \exists\, S : \mathit{Set}(\mathit{Loc}).\ \mathit{regAuth}(S) \\ &\quad * \mathbin{\ast}_{\iota \in S} \mathit{invState}(\iota) * \mathit{disabled}(\top \setminus S) \\ \mathit{regAuth}(S) &\triangleq \mathit{own}\,\gamma_{reg}\,(\bullet\,[\iota \mapsto \mathit{ag}(\mathit{unit})]_{\iota \in S}) \end{aligned}$$

wsat owns the registry, a finite set of allocated invariant names S. *regAuth* asserts exclusive ownership of the registry, similar to *SI*.

Each invariant $\iota \in S$ is in one of two states.

$$\begin{aligned} \mathit{invState}(\iota) \triangleq\ & \big(\exists\, P.\ \mathit{own}\,\iota\,\mathit{ag}(P) * \triangleright P * \mathit{disabled}(\{\iota\})\big) \\ & \vee\ \mathit{enabled}(\{\iota\}) \end{aligned}$$

The first disjunct represents a closed invariant: *wsat* owns its body (under $\triangleright$) together with a *disabled* token, which may be understood as the obligation to close it. The second represents an open invariant, where the body and *disabled* token are absent, and *wsat* has an *enabled* token, indicating that the invariant is currently open. The protocol allows these tokens and resources to move in and out of *wsat*. An invariant cannot be opened while it is already open, since two copies of *enabled* cannot be owned for the same mask. Both tokens are defined for masks E using ownership of disjoint masks, and require two additional dedicated ghost names, γ_{en} and γ_{dis}.

$$\begin{aligned} \mathit{enabled}(E) &\triangleq \mathit{own}\,\gamma_{en}\,\mathit{disjMask}(E) \\ \mathit{disabled}(E) &\triangleq \mathit{own}\,\gamma_{dis}\,\mathit{disjMask}(E) \end{aligned}$$

5.4 User-defined ghost state

Having seen the setup of the concrete heap and invariants, we can now define the cell type mentioned when we first talked about the ghost heap.

As a first approximation, we define it as the sum of the four resource algebras used by the program logic. *InvF* depends on the type P of propositions stored in a cell, so it is a functor. We turn the others into functors using the constant resource algebra functor C, and define:

$$\mathit{BaseCell}(V) \triangleq C(\mathit{Heap}(V)) + \mathit{InvF} + C(\mathit{Reg}) + C(\mathit{Mask})$$

$(\cdot + \cdot)$, a sum of resource algebra functors, is a resource algebra functor. Thus $\mathit{BaseCell}(V)$ is also a resource algebra functor.

This would be sufficient if we only ever worked with the aforementioned kinds of ghost state. However, the motivation for having a rich vocabulary of resource algebras is that users can compose them, to build new problem-specific abstractions, like transition systems or protocols. These would have to be stored in the ghost heap as well. As such, the cell type of the ghost heap should be somehow left extensible.

We do this by following the recipe given in *Data types à la carte* [29]. To illustrate this, suppose the user wants to extend the ghost heap with a proposition-free resource algebra A. A natural first step would be for the user to replace $\mathit{BaseCell}(V)$ with

$$M \triangleq C(A) + \mathit{BaseCell}(V)$$

Assuming the sum is nested to the right, the injection of the invariant resource into $\mathit{BaseCell}(V)$ is

$$p : \mathit{InvF}(P) \longmapsto \mathit{inr}(\mathit{inl}(p)) : \mathit{BaseCell}(V)(P)$$

whereas the corresponding injections into M are

$$\begin{aligned} a : A &\longmapsto \mathit{inl}(a) : M(P) \\ p : \mathit{InvF}(P) &\longmapsto \mathit{inr}(\mathit{inr}(\mathit{inl}(p))) : M(P) \end{aligned}$$

The path for *InvF* changed by one *inr* constructor. This would be a problem if existing client code asserted ownership of an invariant by mentioning the path concretely, as it would then have to be changed to accommodate the extension. The same issue would occur for A's path if we further extended M with another sum.

A better approach would be to abstract over the concrete path by synthesising an injection $\mathit{InvF}(P) \to M(P)$ and proofs of associated laws. Hence, we use a typeclass

[1] A cofinite set is one whose complement is finite. The set of all invariant names is cofinite because its complement is empty.

[2] The *DisjMask* resource algebra allows splitting/joining these sets disjointly.

$InG(M, F)$, with one main operation $inj_P : F(P) \longrightarrow M(P)$, which must be shown to respect the resource algebra structure. The proofs are assembled compositionally by typeclass search.

Clients then only need InG instances to assert that the functors they need are present in an ambient cell type M, rather than requiring M to be exactly $BaseCell(V)$ or some other concrete type. Consequently, for the resource algebra A from earlier, the ownership assertion $own\,\gamma\,a$ requires only the membership $InG(M, C(A))$. The added component may more generally be a resource algebra functor F, allowing user-defined ghost state to contain propositions.

This is one place where our development diverges from Iris, which uses (conceptually) a list of ghost heaps, making ghost names tuples of a list index and a heap location, instead of an extensible sum. We take this route largely for simplicity.

5.5 Weakest preconditions

Following Iris, the weakest precondition is the assertion

$$
\begin{aligned}
&wp_{\mathcal{E}}\ e\ \{\Phi\} \triangleq \forall\,\sigma.\, Res(\sigma) \twoheadrightarrow \Rrightarrow \diamond \\
&\quad \text{if } e = Val(v), \text{ then } Res(\sigma) * \Phi(v); \\
&\quad \text{if } e \notin Val, \text{ then } \diamond\, reducible(e, \sigma) \wedge \\
&\quad \forall\,(e', \sigma', \vec{e}_f) \in step(e, \sigma).\ \Rrightarrow \triangleright \Rrightarrow \diamond \big(Res(\sigma') * wp_{\mathcal{E}}\ e'\ \{\Phi\} \\
&\qquad * \textstyle\ast_{e_f \in \vec{e}_f} wp_{\top}\ e_f\ \{(\lambda_.\, True)\}\big)
\end{aligned}
$$

The resources threaded through the weakest precondition are given by $Res(\sigma) \triangleq SI(\sigma) * wsat * enabled(\mathcal{E})$. If e is a value, the postcondition Φ must hold immediately, while the resources in $Res(\sigma)$ are preserved. Otherwise, e must be reducible, and for every possible new configuration (e', σ'), e' must satisfy the weakest precondition $wp_{\mathcal{E}}\ e'\ \{\Phi\}$ with $Res(\sigma')$ preserved. The list $\vec{e}_f$ contains the threads forked during a reduction step. Each forked thread must satisfy a weakest precondition with postcondition $True$, meaning that it must execute safely. Its return value is discarded.

The argument $\mathcal{E}$ is a mask specifying the invariants that may be opened during the execution of the expression. It must be restored after each physical step. Forked threads begin with no invariants open and therefore use the full mask $\top$. The $\diamond$s waive the obligations in the reducible case at index 0, where there are no steps remaining and we cannot distinguish stuck expressions from reducible ones. The $\Rrightarrow \triangleright \Rrightarrow$ allows ghost state updates both before and after a logical step. A common example is proving that an invariant is preserved by an atomic step: the first basic update opens the invariant and obtains its body under $\triangleright$, the physical step consumes the $\triangleright$, and the second update can then reestablish and close the invariant. The $\triangleright$ both synchronises program steps with logical steps and guards the recursion.

We may then define the Hoare triple as

$$\{P\}\ e\ \{\Phi\}_{\mathcal{E}} \triangleq \Box\big(P \twoheadrightarrow wp_{\mathcal{E}}\ e\ \{\Phi\}\big)$$

and use it to derive all the standard program logic rules.

5.6 Verified programs

We validate the program logic on simple but characteristic examples. First, parallel increment, where two threads perform a non-atomic increment on a shared location. We prove specifications of varying strength using different techniques, including a plain invariant, and a three-state protocol using a combination of the one-shot and fraction resource algebras.

Second, we verify Landin's Knot [21]: both by ordinary induction and by Löb induction [8], where an invariant owns the self-referential cell. The resulting proof exercises impredicative invariants, persistent higher-order specifications, and guarded recursion together.

5.7 Discussion

This case study validates the claim that Indirection Theory can serve as an alternative foundation for Iris. To this end, the construction of the program logic intentionally follows Iris closely, to sufficient depth that we can reconstruct a program logic usable for verifying simple programs.

The proofs are written using Lean tactics for rewriting logical entailments, permuting separating conjunctions, frame inference, biabduction, and ramified Hoare triples. They therefore resemble conventional [10] interactive separation-logic proofs; an equivalent to the Iris Proof Mode [19] is not implemented. Reuse of the Iris-Lean implementation of it [12] is likely to be possible, given that our separation logic is a standard affine BI logic.

6 Related Work

From step-indexing to higher-order separation logic. Step-indexing originated as a technique for modelling recursive types [5] and higher-order state [1]. Appel et al. [6] then abstracted it into the $\triangleright$ operator of a modal logic, a presentation that later works [13] established as a standard way of handling step-indexing. Hobor et al. [15] used this approach to build a higher-order concurrent separation logic, later generalising the construction to Indirection Theory [16, 4]. Meanwhile, Birkedal et al. [9] developed a different construction, based on ultrametric spaces, and used the theorem of America and Rutten [2] to solve recursive domain equations. This construction, adapted to (complete) ordered families of equivalences and (unital) cameras, is the foundation of Iris [17].

Indirection Theory. Indirection Theory was introduced by Hobor et al. [16] and mechanised in Rocq in the Mechanized Semantic Library (MSL) [4]. MSL was later extended and used as the foundation of VST [3]. We mostly follow the MSL development, with the main difference being that we replace separation algebras with Iris-style resource algebras. Additionally, MSL supports some minor generalisations beyond our presentation. For example, MSL supports functors of mixed variance, whereas ours are covariant. These

extensions are straightforward and not central to our development, so we omit them for ease of implementation and presentation.

PulseCore. The closest system to ours is PulseCore [14], an impredicative concurrent separation logic embedded in F⋆. It also uses Indirection Theory as a foundation for reasons of simplicity, and arrives at a very similar "base logic" of hereditary predicates, with two main differences.

The first is in the construction of the knot. PulseCore recursively defines the resources rather than the propositions, and a stored proposition is a dependent function over knots of lower level rather than a sequence of approximations. They report that F⋆'s extensional type theory makes this a shorter construction; the advantage does not carry over to Lean's intensional type theory, so we follow the classic approach of defining propositions recursively.

A second difference is in what the knot is instantiated with. PulseCore fixes the higher-order ghost state to a heap whose cells hold only either an invariant or a stored predicate. First-order ghost state is contained in a separate heap and other higher-order ghost state is obtained by encoding. Its authors note that they could "make the construction parametric in the supported higher-order ghost state cells and their associated functors", but decided against "the API complexity of extra parameters". This is the generalisation we make, in parameterising the knot by an arbitrary resource algebra functor, so higher-order ghost state is user-definable, like in Iris.

Other practical differences in PulseCore outside of its model construction are that it is meant for intrinsic proofs about F⋆ programs, so the step index is decreased by ghost code rather than by steps of the reduction relation, and a Löb induction principle is not exposed at all, since F⋆'s native well-founded recursion suffices.

VST. VST [3] is a program logic for C, and the most substantial system built on Indirection Theory. VST's resource model is closely tied to CompCert [22]'s memory model, so that the program logic can be proven sound against CompCert's operational semantics. Verification in VST then combines with CompCert's compiler correctness to give guarantees about compiled code. In particular, VST's resource is a heap, in which a location holds either a value with a permission, or a predicate, such as a function's specification or a lock's invariant.

Mansky [23] brought some of Iris's ghost state into VST to increase its expressiveness, while keeping its resource model tied to CompCert. Ghost state was added as a second component of the resource

$$heap \times ghost$$

and invariants were derived from it in the same work. However, this approach still could not fully support higher-order ghost state, due to universe inconsistencies.

This limitation motivated Mansky and Du [24] to rebuild VST's program logic on top of Iris, from which higher-order ghost state is then inherited. Both works left open the question of whether Indirection Theory can be made as expressive as Iris. Our work addresses this question.

Iris and Iris-Lean. Iris [17] is the frontier of higher-order concurrent separation logic, with a large body of features and applications built on it, and is mechanised in Rocq. Iris-Lean [18, 12] is a faithful port of Iris to Lean, and, to our knowledge, the only other mechanised step-indexed logic in Lean. It follows Iris's construction directly, up to minor differences between the two proof assistants, so the comparison to Iris's base logic in §2 and §3 applies to it unchanged.

7 Conclusion

Our aim in this work was to clarify and generalise the foundations of a higher-order separation logic, thereby making the construction of higher-order semantic models and program logics accessible to a wider audience, and spurring the development of more expressive verification tools. To this end, we presented a Lean implementation of Indirection Theory, generalised to support Iris-style higher-order ghost state. Its construction is elementary and uses only dependent types, without domain theory or category theory. It should thus serve as a recipe for implementing similar models. We demonstrated the expressiveness of the resulting separation logic for constructing both logical relations and an Iris-style program logic, and also clarified the relationship between Indirection Theory and Iris's model.

Some parts of Iris remain to be reconstructed. The most immediate direction for future work is to reconstruct the recent work on discharging ▷ on these foundations: later credits [28], and the physical-step modality [27].

References

[1] Amal J. Ahmed, Andrew W. Appel, and Roberto Virga. 2002. A stratified semantics of general references. In *17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings*. IEEE Computer Society, 75. doi:10.1109/LICS.2002.1029818.

[2] Pierre America and Jan J. M. M. Rutten. 1989. Solving reflexive domain equations in a category of complete metric spaces. *J. Comput. Syst. Sci.*, 39, 3, 343–375. doi:10.1016/0022-0000(89)90027-5.

[3] Andrew W. Appel. 2012. Verified software toolchain. In *NFM* (LNCS). Alwyn Goodloe and Suzette Person, (Eds.) Vol. 7226. Springer, 2. doi:10.1007/978-3-642-28891-3_2.

[4] Andrew W. Appel, Robert Dockins, and Aquinas Hobor. 2010. Mechanized semantic library. https://vst.cs.princeton.edu/msl/. (2010).

[5] Andrew W. Appel and David A. McAllester. 2001. An indexed model of recursive types for foundational proof-carrying code. *ACM Trans. Program. Lang. Syst.*, 23, 5, 657–683. doi:10.1145/504709.504712.

[6] Andrew W. Appel, Paul-André Melliès, Christopher D. Richards, and Jérôme Vouillon. 2007. A very modal model of a modern, major, general type system. In *POPL*. Martin Hofmann and Matthias Felleisen, (Eds.) ACM, 109–122. doi:10.1145/1190216.1190235.

[7] Patrycja Balik, Dariusz Biernacki, and Piotr Polesiuk. 2025. Untyped logical relations at work: control operators, contextual equivalence and full abstraction. In *Proceedings of the Workshop Dedicated to Olivier Danvy on the Occasion of His 64th Birthday* (OLIVIERFEST '25). Association for Computing Machinery, Singapore, Singapore, 128–141. doi:10.1145/3759427.3760374.

[8] Lars Birkedal and Aleš Bizjak. 2023. Lecture notes on Iris: higher-order concurrent separation logic. *Lectures notes, September*. https://iris-project.org/tutorial-material.html.

[9] Lars Birkedal, Kristian Støvring, and Jacob Thamsborg. 2010. The category-theoretic solution of recursive metric-space equations. *Theor. Comput. Sci.*, 411, 47, 4102–4122. doi:10.1016/J.TCS.2010.07.010.

[10] Arthur Charguéraud. 2024. *Separation Logic Foundations*.

[11] Leonardo de Moura and Sebastian Ullrich. 2021. The lean 4 theorem prover and programming language. In *Automated Deduction - CADE 28 - 28th International Conference on Automated Deduction, Virtual Event, July 12-15, 2021, Proceedings* (Lecture Notes in Computer Science). André Platzer and Geoff Sutcliffe, (Eds.) Vol. 12699. Springer, 625–635. doi:10.1007/978-3-030-79876-5_37.

[12] Iris-Lean developers. 2026. Iris-lean. https://github.com/leanprover-community/iris-lean. (2026).

[13] Derek Dreyer, Amal Ahmed, and Lars Birkedal. 2009. Logical step-indexed logical relations. In *LICS*. IEEE Computer Society, 71–80. doi:10.1109/LICS.2009.34.

[14] Gabriel Ebner, Guido Martínez, Aseem Rastogi, Thibault Dardinier, Megan Frisella, Tahina Ramananandro, and Nikhil Swamy. 2025. Pulsecore: an impredicative concurrent separation logic for dependently typed programs. In *PLDI*. ACM. https://www.microsoft.com/en-us/research/publication/pulsecore-an-impredicative-concurrent-separation-logic-for-dependently-typed-programs/.

[15] Aquinas Hobor, Andrew W. Appel, and Francesco Zappa Nardelli. 2008. Oracle semantics for concurrent separation logic. In *Programming Languages and Systems, 17th European Symposium on Programming, ESOP 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings* (Lecture Notes in Computer Science). Sophia Drossopoulou, (Ed.) Vol. 4960. Springer, 353–367. doi:10.1007/978-3-540-78739-6_27.

[16] Aquinas Hobor, Robert Dockins, and Andrew W. Appel. 2010. A theory of indirection via approximation. In *POPL*. Manuel V. Hermenegildo and Jens Palsberg, (Eds.) ACM, 171–184. doi:10.1145/1706299.1706322.

[17] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.*, 28, e20. doi:10.1017/S0956796818000151.

[18] Lars König. 2022. *An Improved Interface for Interactive Proofs in Separation Logic*. Ph.D. Dissertation. Karlsruher Institut für Technologie (KIT).

[19] Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. 2018. Mosel: A general, extensible modal framework for interactive proofs in separation logic. *Proc. ACM Program. Lang.*, 2, ICFP, 77:1–77:30. doi:10.1145/3236772.

[20] Robbert Krebbers, Ralf Jung, Ales Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. 2017. The essence of higher-order concurrent separation logic. In *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings* (Lecture Notes in Computer Science). Hongseok Yang, (Ed.) Vol. 10201. Springer, 696–723. doi:10.1007/978-3-662-54434-1_26.

[21] P. J. Landin. 1964. The mechanical evaluation of expressions. *Comput. J.*, 6, 4, 308–320. doi:10.1093/COMJNL/6.4.308.

[22] Xavier Leroy, Sandrine Blazy, Daniel Kästner, Bernhard Schommer, Markus Pister, and Christian Ferdinand. 2016. CompCert – a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems*. SEE.

[23] William Mansky. 2022. Bringing iris into the verified software toolchain. *CoRR*, abs/2207.06574. arXiv: 2207.06574. doi:10.48550/ARXIV.2207.06574.

[24] William Mansky and Ke Du. 2024. An iris instance for verifying compcert C programs. *Proc. ACM Program. Lang.*, 8, POPL, 148–174. doi:10.1145/3632848.

[25] Peter W. O'Hearn. 2007. Resources, concurrency, and local reasoning. *Theor. Comput. Sci.*, 375, 1-3, 271–307. doi:10.1016/J.TCS.2006.12.035.

[26] Piotr Polesiuk. 2017. Ixfree: step-indexed logical relations in coq. In *CoqPL*.

[27] Thomas Somers, Jonas Kastberg Hinrichsen, Lennard Gäher, and Robbert Krebbers. 2026. Building blocks for step-indexed program logics. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2026, Rennes, France, January 12-13, 2026*. Kathrin Stark, Yannick Zakowski, Nikhil Swamy, and Nicolas Tabareau, (Eds.) ACM, 248–263. doi:10.1145/3779031.3779095.

[28] Simon Spies, Lennard Gäher, Joseph Tassarotti, Ralf Jung, Robbert Krebbers, Lars Birkedal, and Derek Dreyer. 2022. Later credits: resourceful reasoning for the later modality. *Proc. ACM Program. Lang.*, 6, ICFP, 283–311. doi:10.1145/3547631.

[29] Wouter Swierstra. 2008. Data types à la carte. *J. Funct. Program.*, 18, 4, 423–436. doi:10.1017/S0956796808006758.